

Agente Explorador

Projeto de Programação III

Grupo: Grupo G

Aluno: Pedro Manuel Domingos Portinha

Número: 28501

Data:08/07/2026

Índice

1. Introdução	3
2. Ferramentas utilizadas.....	3
3. Desenvolvimento do projeto	4
3.1 Organização inicial do projeto	4
3.2 Implementação da comunicação com a Arena	5
3.3 Desenvolvimento do Motor Heurístico	6
3.4 Melhoria da lógica de combate.....	7
3.5 Desenvolvimento do mapa de calor.....	8
3.6 Integração da componente RAG	9
4. Utilização do Codex	10
5. Resumo da utilização da IA	11
6. Conclusão.....	12
7. Anexos	12

1. Introdução

Durante o desenvolvimento deste projeto utilizei principalmente o ChatGPT e, numa fase final, o Codex, como ferramentas de apoio. A utilização destas ferramentas foi evoluindo ao longo do projeto. Numa fase inicial serviram para esclarecer dúvidas sobre a estrutura do projeto e sobre a comunicação com a Arena. Mais tarde passaram a ser utilizadas para melhorar a heurística do robô, corrigir comportamentos observados durante os testes e validar pequenas alterações ao código.

Sempre que surgia um problema procurava primeiro perceber a sua origem e só depois recorria à IA para discutir possíveis soluções. As respostas nunca foram aplicadas diretamente. Todas as sugestões foram adaptadas ao contexto do projeto e testadas na Arena antes de serem integradas.

O objetivo deste documento é apresentar as principais utilizações da Inteligência Artificial durante o desenvolvimento do projeto e explicar de que forma essas interações contribuíram para a solução final.

2. Ferramentas utilizadas

ChatGPT

Foi utilizado durante praticamente todo o desenvolvimento para:

- esclarecer dúvidas;
- interpretar o enunciado;
- ajudar na implementação de funcionalidades;
- melhorar a heurística;
- resolver problemas encontrados durante os testes.

Codex

Foi utilizado na fase final do projeto para rever código existente, validar alterações, sugerir pequenas melhorias e confirmar o cumprimento dos requisitos do enunciado

3. Desenvolvimento do projeto

3.1 Organização inicial do projeto

Contexto

Antes de começar a desenvolver o agente foi necessário criar a estrutura do projeto e preparar o ambiente de desenvolvimento.

Prompt:

“Qual é a melhor forma de organizar um projeto Maven para cumprir os requisitos do enunciado?”

Resposta da IA

A IA sugeriu criar um projeto Maven organizado por responsabilidades, separando as diferentes componentes em packages distintos.

Foram também apresentadas recomendações relativamente à criação do pom.xml, organização do código, utilização do Git e configuração do ficheiro .gitignore.

Além disso, foi sugerido separar a comunicação com a Arena, a heurística, a componente RAG e a interface gráfica em classes independentes.

Como utilizei esta sugestão:

A IA sugeriu dividir o projeto por responsabilidades, separando a comunicação com a Arena, o motor heurístico, a componente RAG e a interface gráfica em packages distintos. Essa organização foi utilizada como ponto de partida para o desenvolvimento e manteve-se praticamente inalterada até ao final do projeto.

3.2 Implementação da comunicação com a Arena

Contexto

Depois de criada a estrutura do projeto, o passo seguinte consistiu em implementar a comunicação entre o robô e a Arena.

Antes de escrever qualquer código foi necessário compreender a documentação disponibilizada e perceber como funcionavam os diferentes endpoints da API.

Prompt

"Como devo implementar a comunicação entre o agente e a Arena utilizando a API disponibilizada?"

Resposta da IA

Foi sugerido criar uma classe específica para toda a comunicação com a Arena.

A IA explicou a utilização do HttpClient para efetuar pedidos HTTP e da biblioteca Gson para converter automaticamente os objetos Java para JSON.

Foram igualmente sugeridas algumas boas práticas, nomeadamente:

- reutilização do mesmo objeto HttpClient;
- tratamento de exceções;
- validação dos códigos de resposta;
- separação da comunicação do restante código.

Como utilizei esta sugestão:

A IA recomendou concentrar toda a comunicação numa única classe, utilizando HttpClient para os pedidos HTTP e Gson para o processamento das mensagens JSON. Com base nessa sugestão desenvolvi o ArenaClient, responsável pelo registo do robô, obtenção da perceção, envio de ações e abertura de cofres.

3.3 Desenvolvimento do Motor Heurístico

Contexto

Depois de implementar a comunicação com a Arena, comecei a testar o comportamento do robô. Embora já conseguisse explorar o mapa e executar as ações básicas, rapidamente percebi que a lógica de decisão era demasiado simples. Durante os testes surgiram vários problemas: o robô repetia frequentemente os mesmos movimentos, nem sempre atribuía prioridade aos cofres e recolhia recursos mesmo quando existiam objetivos mais importantes.

Prompt

"O robô já funciona, mas continua a tomar más decisões na exploração, abertura de cofres e combate. Como posso melhorar a heurística?"

Resposta da IA

A IA sugeriu várias melhorias para tornar a tomada de decisão mais consistente, entre as quais:

- criar memória das posições visitadas;
- memorizar cofres encontrados;
- definir prioridades entre exploração, recursos e cofres;
- evitar movimentos repetitivos;
- ajustar a lógica utilizada durante os combates.

Como utilizei esta sugestão

Em vez de alterar toda a implementação de uma só vez, fui introduzindo pequenas melhorias e testando cada uma delas na Arena. A heurística passou a considerar o histórico das posições visitadas, a existência de cofres conhecidos e a prioridade dos recursos de acordo com a energia disponível.

Ao longo dos testes continuei a recorrer ao ChatGPT para corrigir comportamentos específicos que iam surgindo. Algumas dessas interações permitiram melhorar a exploração do mapa, reduzindo movimentos repetitivos e incentivando a descoberta de novas zonas. Outras ajudaram a reorganizar a gestão dos cofres, criando uma memória dos objetivos já conhecidos e evitando deslocações desnecessárias. Também foram ajustadas as prioridades na recolha de recursos, passando estes a ser procurados apenas quando realmente necessários.

No final desta fase, o comportamento do robô tornou-se significativamente mais consistente, explorando melhor o mapa, gerindo os objetivos de forma mais eficiente e reduzindo decisões pouco vantajosas observadas nos primeiros testes.

3.4 Melhoria da lógica de combate

Contexto

Depois de melhorar a exploração do mapa, comecei a reparar que o robô continuava a tomar decisões pouco eficazes quando encontrava outros agentes.

Durante alguns testes atacava adversários claramente mais fortes e acabava por perder o combate. Noutras situações fazia exatamente o contrário, fugindo mesmo quando tinha vantagem.

Prompt

"O robô continua a tomar más decisões durante os combates. Como posso melhorar esta parte da heurística?"

Resposta da IA

A principal sugestão foi deixar de decidir apenas com base na distância ao inimigo.

Em vez disso, a decisão deveria considerar vários fatores em simultâneo, como:

- energia atual do robô;
- energia estimada do adversário;
- possibilidade de recolher energia antes do combate;
- existência de um caminho seguro para fugir;
- importância do objetivo que estava a ser perseguido.

Foi ainda sugerido que, sempre que existisse uma diferença significativa de energia, o agente deveria adaptar automaticamente o seu comportamento.

Como utilizei esta sugestão:

Em algumas situações o robô atacava adversários claramente mais fortes e, noutras, evitava confrontos quando tinha vantagem.

A IA sugeriu que a decisão deixasse de depender apenas da proximidade do inimigo e passasse também a considerar fatores como a energia disponível, a distância ao adversário e o contexto em que o combate ocorria.

Estas ideias foram integradas no Motor Heurístico e ajustadas após vários testes na Arena. Como resultado, o robô passou a selecionar melhor os combates e a evitar confrontos desnecessários.

3.5 Desenvolvimento do mapa de calor

Contexto

À medida que o Motor Heurístico se tornava mais complexo, começou a ser cada vez mais difícil perceber porque motivo o robô tomava determinadas decisões.

Prompt

"O mapa de calor ainda não representa corretamente os inimigos e algumas zonas exploradas"

Resposta da IA

A IA sugeriu transformar o mapa de calor numa ferramenta de apoio ao desenvolvimento.

Além da intensidade das visitas, foram sugeridas novas representações para:

- posição atual do robô;
- inimigos visíveis;
- recursos conhecidos;
- cofres encontrados;
- paredes descobertas.

Foi igualmente sugerido atualizar automaticamente o painel sempre que o agente recebesse uma nova perceção da Arena.

Como utilizei esta sugestão:

À medida que o Motor Heurístico se tornou mais complexo, passou a ser mais difícil perceber porque motivo o robô tomava determinadas decisões.

Para facilitar essa análise foi desenvolvido um mapa de calor que, numa primeira versão, apenas representava o número de visitas realizadas a cada posição.

Posteriormente, com o apoio das sugestões obtidas através da IA, o painel foi sendo melhorado para representar também inimigos, recursos, cofres, paredes e a posição atual do robô.

3.6 Integração da componente RAG

Contexto

Uma das funcionalidades pedidas no enunciado consistia na utilização de um sistema **Retrieval-Augmented Generation (RAG)** para ajudar o robô a responder às perguntas dos cofres.

Esta foi uma das partes do projeto onde tive menos experiência inicial, principalmente por envolver conceitos relacionados com modelos de linguagem e recuperação de informação.

Prompt

“Já tenho o Ollama a funcionar. Como posso ligar isto a um sistema RAG para responder aos cofres?”

Resposta da IA:

A IA explicou a arquitetura geral de um sistema RAG, sugerindo separar a pesquisa de informação da geração da resposta e utilizar o Ollama para produzir a resposta final com base no contexto recuperado.

Como utilizei esta sugestão:

A implementação da componente RAG foi uma das fases em que utilizei a IA para compreender conceitos que ainda não dominava completamente.

As respostas obtidas permitiram perceber a arquitetura geral de um sistema RAG, a utilização de embeddings e a recuperação de contexto antes da geração da resposta.

Com base nessas sugestões implementei a comunicação com o Ollama e a recuperação da informação utilizada na resolução dos desafios apresentados pelos cofres.

4. Utilização do Codex

Contexto

O Codex foi utilizado principalmente na fase final do desenvolvimento. Ao contrário do ChatGPT, começava por analisar automaticamente o projeto, abrir as classes relevantes e compreender a estrutura existente antes de sugerir alterações. Depois de cada modificação eram efetuadas novas compilações para confirmar que o projeto continuava funcional.

Revisão do projeto

Antes de sugerir alterações, o Codex analisava automaticamente a estrutura do projeto, identificando as classes mais relevantes e verificando o estado da implementação. Só depois dessa análise eram propostas pequenas melhorias ao código existente.

Melhorias efetuadas

Durante esta fase o Codex foi utilizado principalmente para:

- rever o Motor Heurístico;
- melhorar o PainelMapaCalor;
- corrigir pequenos erros encontrados durante os testes;
- validar o cumprimento dos requisitos do enunciado;
- sugerir pequenas otimizações ao código existente.

Ao contrário das primeiras fases do desenvolvimento, nesta etapa praticamente todas as alterações eram pequenas e bastante localizadas.

Depois de cada alteração o projeto era novamente compilado para confirmar que continuava funcional.

Compilação e validação

Um aspeto que considerei particularmente útil foi o facto de o Codex validar frequentemente as alterações através da compilação do projeto.

Sempre que eram efetuadas alterações significativas, o projeto era compilado novamente para verificar a existência de erros.

Esta abordagem permitiu identificar rapidamente pequenos problemas introduzidos durante as alterações, facilitando bastante o processo de correção.

5. Resumo da utilização da IA

A tabela seguinte resume as principais fases do projeto, a ferramenta de IA utilizada e a forma como contribuiu para o desenvolvimento.

Fase do projeto	Ferramenta	Principal utilização
Organização inicial do projeto	ChatGPT	Definição da estrutura do projeto e organização em packages.
Comunicação com a Arena	ChatGPT	Apoio na implementação do ArenaClient e da comunicação REST.
Motor Heurístico	ChatGPT	Melhoria da exploração, combate, gestão de cofres e redução de loops.
Mapa de calor	ChatGPT	Sugestões para melhorar a visualização e apoiar a análise dos testes.
Componente RAG	ChatGPT	Explicação da arquitetura RAG e integração com o Ollama.
Revisão e otimização	Codex	Revisão do código, pequenas melhorias, compilação e validação final.

6. Conclusão

Considero que a utilização destas ferramentas foi útil ao longo do projeto, principalmente porque permitiu esclarecer dúvidas e acelerar a resolução de alguns problemas. No entanto, todas as sugestões tiveram de ser analisadas e testadas antes de serem integradas, uma vez que nem sempre funcionavam corretamente no contexto da Arena. Ao longo do desenvolvimento também percebi que a qualidade das respostas dependia da forma como os problemas eram descritos, o que me levou a formular pedidos cada vez mais específicos e úteis.

7. Anexos

Anexo A – Histórico de conversas do ChatGPT

O histórico completo das interações realizadas com o ChatGPT durante o desenvolvimento do projeto encontra-se disponível através do seguinte link:

<https://chatgpt.com/share/6a4e6fcc-86d4-83eb-934e-226900961566>

Anexo B – Histórico de interações com o Codex

O histórico das interações realizadas com o Codex durante a fase final do desenvolvimento encontra-se incluído no repositório GitHub através do ficheiro:

conversasCodex.txt

Este ficheiro reúne as principais interações realizadas com o Codex durante a fase final do desenvolvimento, utilizadas para revisão do código, pequenas melhorias e validação do projeto.